Map Reduce (2)

Question 5 Ecrivez le programme MapReduce correspondant à l'étape MR1.

Question 6 Ecrivez le programme MapReduce correspondant à l'étape MR2.

Question 7 Ecrivez le programme MapReduce correspondant à l'étape MR3.

Question 8 Ecrivez le programme MapReduce correspondant à l'étape MR4.

Codes déjà fournis

Question 9 Ecrivez le programme complet. Vous écrirez un programme TfIdf qui appellera successivement MR1, MR2, MR3 et MR4. Le(s) fichier(s) résultant de l'un étant le(s) fichier(s) d'entrée du suivant.

```java
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.mapreduce.lib.input.KeyValueTextInputFormat;

public class TfIdf {
    static Configuration conf;

    public static void main(String[] args) throws Exception {
        TfIdf pipeline = new TfIdf();
        conf = new Configuration();
        FileSystem fs = FileSystem.get(conf);

        String entree = args[0];
        String sortie = args[1];
        int nombreDocuments = Integer.parseInt(args[2]);

        // Lancement du premier MapReduce
        fs.delete(new Path(sortie + "_1"), true);
        pipeline.lanceEtape1(entree, sortie + "_1");

        // Lancement du second MapReduce
        fs.delete(new Path(sortie + "_2"), true);
        pipeline.lanceEtape2(sortie + "_1", sortie + "_2");

        // Lancement du troisième MapReduce
        fs.delete(new Path(sortie + "_3"), true);
        pipeline.lanceEtape3(sortie + "_2", sortie + "_3");

        // Lancement du quatrième MapReduce
        fs.delete(new Path(sortie + "_4"), true);
        pipeline.lanceEtape4(sortie + "_3", sortie + "_4", nombreDocuments);
    }
```

```java

    public int lanceEtape1(String entree, String sortie) throws Exception {
        Job job = new Job(conf);
        job.setJarByClass(TfIdf.class);
        job.setMapperClass(MR1.MR1Mapper.class);
        job.setReducerClass(MR1.MR1Reducer.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(entree));
        FileOutputFormat.setOutputPath(job, new Path(sortie));
        boolean success = job.waitForCompletion(true);
        return success ? 0 : -1;
    }

    public int lanceEtape2(String entree, String sortie) throws Exception {
        Job job = new Job(conf);
        job.setJarByClass(TfIdf.class);
        job.setMapperClass(MR2.MR2Mapper.class);
        job.setReducerClass(MR2.MR2Reducer.class);
        job.setInputFormatClass(KeyValueTextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        FileInputFormat.addInputPath(job, new Path(entree));
        FileOutputFormat.setOutputPath(job, new Path(sortie));
        boolean success = job.waitForCompletion(true);
        return success ? 0 : -1;
    }

```

```java

    public int lanceEtape3(String entree, String sortie) throws Exception {
        Job job = new Job(conf);
        job.setJarByClass(TfIdf.class);
        job.setMapperClass(MR3.MR3Mapper.class);
        job.setReducerClass(MR3.MR3Reducer.class);
        job.setInputFormatClass(KeyValueTextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        FileInputFormat.addInputPath(job, new Path(entree));
        FileOutputFormat.setOutputPath(job, new Path(sortie));
        boolean success = job.waitForCompletion(true);
        return success ? 0 : -1;
    }

    public int lanceEtape4(String entree, String sortie, int nombreDocuments) throws Exception {
        conf.setDouble("NombreDocs", nombreDocuments);
        Job job = new Job(conf);
        job.setJarByClass(MR4.class);
        job.setMapperClass(MR4.MR4Mapper.class);
        job.setInputFormatClass(KeyValueTextInputFormat.class);
        // job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        FileInputFormat.addInputPath(job, new Path(entree));
        FileOutputFormat.setOutputPath(job, new Path(sortie));
        boolean success = job.waitForCompletion(true);
        return success ? 0 : -1;
    }
}
```

Résultat du programme : le résultat de chaque étape est sauvegardé dans un fichier différent, le résultat de l'étape 4 est celui qui nous intéresse :

```
gaga|hdfs://cloud-41.depinfo.etu:9000/rep/3     1|5
tutu|hdfs://cloud-41.depinfo.etu:9000/rep/3     1|5
tata|hdfs://cloud-41.depinfo.etu:9000/rep/3     2|5
toto|hdfs://cloud-41.depinfo.etu:9000/rep/3     1|5
titi|hdfs://cloud-41.depinfo.etu:9000/rep/4     1|6
tata|hdfs://cloud-41.depinfo.etu:9000/rep/4     4|6
gogo|hdfs://cloud-41.depinfo.etu:9000/rep/4     1|6
etudiant@cloud-41:~/tp5$ hadoop fs -cat /tfidf_3/part-r-00000
gaga|hdfs://cloud-41.depinfo.etu:9000/rep/3     1|5|2
gaga|hdfs://cloud-41.depinfo.etu:9000/rep/1     1|6|2
gogo|hdfs://cloud-41.depinfo.etu:9000/rep/4     1|6|1
tata|hdfs://cloud-41.depinfo.etu:9000/rep/3     2|5|3
tata|hdfs://cloud-41.depinfo.etu:9000/rep/4     4|6|3
tata|hdfs://cloud-41.depinfo.etu:9000/rep/2     1|4|3
tete|hdfs://cloud-41.depinfo.etu:9000/rep/2     1|4|1
titi|hdfs://cloud-41.depinfo.etu:9000/rep/4     1|6|3
titi|hdfs://cloud-41.depinfo.etu:9000/rep/2     1|4|3
titi|hdfs://cloud-41.depinfo.etu:9000/rep/1     2|6|3
toto|hdfs://cloud-41.depinfo.etu:9000/rep/3     1|5|3
toto|hdfs://cloud-41.depinfo.etu:9000/rep/1     2|6|3
toto|hdfs://cloud-41.depinfo.etu:9000/rep/2     1|4|3
tutu|hdfs://cloud-41.depinfo.etu:9000/rep/3     1|5|2
tutu|hdfs://cloud-41.depinfo.etu:9000/rep/1     1|6|2
etudiant@cloud-41:~/tp5$ hadoop fs -cat /tfidf_4/part-r-00000
gaga|hdfs://cloud-41.depinfo.etu:9000/rep/1     0.11552453009332421
gaga|hdfs://cloud-41.depinfo.etu:9000/rep/3     0.13862943611198905
gogo|hdfs://cloud-41.depinfo.etu:9000/rep/4     0.23104906018664842
tata|hdfs://cloud-41.depinfo.etu:9000/rep/2     0.07192051811294521
tata|hdfs://cloud-41.depinfo.etu:9000/rep/3     0.11507282898071235
tata|hdfs://cloud-41.depinfo.etu:9000/rep/4     0.19178804830118723
tete|hdfs://cloud-41.depinfo.etu:9000/rep/2     0.34657359027997264
titi|hdfs://cloud-41.depinfo.etu:9000/rep/1     0.09589402415059362
titi|hdfs://cloud-41.depinfo.etu:9000/rep/2     0.07192051811294521
titi|hdfs://cloud-41.depinfo.etu:9000/rep/4     0.04794701207529681
toto|hdfs://cloud-41.depinfo.etu:9000/rep/1     0.09589402415059362
toto|hdfs://cloud-41.depinfo.etu:9000/rep/2     0.07192051811294521
toto|hdfs://cloud-41.depinfo.etu:9000/rep/3     0.05753641449035617
tutu|hdfs://cloud-41.depinfo.etu:9000/rep/1     0.11552453009332421
tutu|hdfs://cloud-41.depinfo.etu:9000/rep/3     0.13862943611198905
etudiant@cloud-41:~/tp5$
```

Dans cette image, on voit bien le résultat de l'étape 4 du tf-idf, ainsi que l'étape 3.

Question 10

1. **Réfléchissez à la manière dont vous implémenterez un grep sur un mot donné dans un fichier quelconque. Rappel : grep retourne toutes les lignes dans lequel apparait le mot demandé.**

```java

public class Grep {
    public static class GrepMapper extends Mapper<Text, Text, Text, Text> {
        private Text motRecherche;

        //@Override
        protected void setup(Context context) throws IOException, InterruptedException {
            motRecherche = new Text(context.getConfiguration().get("motRecherche"));
        }

        //@Override
        protected void map(Text key, Text value, Context context) throws IOException, InterruptedException {
            String ligne = key.toString();
            if (ligne.contains(motRecherche.toString())){
                context.write(key, new Text());
            }
        }
    }


    public static class GrepReducer extends Reducer<Text, Text, Text, Text> {
        @Override
        public void reduce(Text clef, Iterable<Text> valeurs, Context context) throws IOException, InterruptedExcepti
            for (Text valeur : valeurs){
                context.write(clef, valeur);
            }
        }

    }
```

```java

    public static void main(String args[]) throws Exception {
        if (args.length != 3) {
            System.err.println(args.length + "(" + args[0] + "," + args[1] + ")");
            System.err.  println("Usage : Grep <source> <destination> mot");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        conf.set("motRecherche", args[2]);

        Job job = Job.getInstance(conf, "Grep");
        job.setJarByClass(Grep.class);

        job.setInputFormatClass(KeyValueTextInputFormat.class);

        job.setMapperClass(GrepMapper.class);
        job.setReducerClass(GrepReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

<u>Résultat</u> : on test avec le grep avec le mot toto, on obtient les lignes des fichiers content ce mot :

```
etudiant@cloud-41:~/tp5$ hadoop jar Grep.jar Grep /rep/ /grep_res/ toto
```

```
etudiant@cloud-41:~/tp5$ hadoop fs -cat /grep_res/part-r-00000
toto
toto
toto
toto
etudiant@cloud-41:~/tp5$
```

2. **Ecrivez un programme qui prend un mot en argument, puis à partir de votre fichier d'indexation tf – idf généré précédemment, sort tous les noms de fichiers le contenant ainsi que les valeurs tf – idf respectifs.**

```java
import java.io.IOException;

public class SearchTFIDF {

    public static class SearchMapper extends Mapper<Object, Text, Text, Text> {

        private String motRecherche;

        @Override
        protected void setup(Context context) throws IOException, InterruptedException {
            motRecherche = context.getConfiguration().get("motRecherche");
        }

        @Override
        public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
            String ligne = value.toString();
            String[] parties = ligne.split("\\s+");
            String motFichier = parties[0].split("\\|")[0];
            if (motFichier.equals(motRecherche)) {
                String nomFichier = parties[0].split("\\|")[1];
                String tfidf = parties[1];
                context.write(new Text(nomFichier), new Text(tfidf));
            }
        }
    }

    public static class SearchReducer extends Reducer<Text, Text, Text, Text> {
        @Override
        public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedExceptio
            for (Text value : values) {
                context.write(key, value);
            }
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        conf.set("motRecherche", args[0]);
        Job job = Job.getInstance(conf, "SearchTFIDF");
        job.setJarByClass(SearchTFIDF.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapperClass(SearchMapper.class);
        job.setReducerClass(SearchReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        TextInputFormat.addInputPath(job, new Path(args[1]));
        TextOutputFormat.setOutputPath(job, new Path(args[2]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

<u>Résultat</u> : On recherche le mot toto en utilisant le fichier généré grâce au programme tf-idf de la question précédente, on utilise le résultat de l'étape 4 :

```
etudiant@cloud-41:~/tp5$ hadoop jar SearchTFIDF.jar SearchTFIDF toto /tfidf_4/part-r-00000 /searchWord
```

On obtient ce résultat :

```
etudiant@cloud-41:~/tp5$ hadoop fs -cat /searchWord/part-r-00000
hdfs://cloud-41.depinfo.etu:9000/rep/1  0.09589402415059362
hdfs://cloud-41.depinfo.etu:9000/rep/2  0.07192051811294521
hdfs://cloud-41.depinfo.etu:9000/rep/3  0.05753641449035617
etudiant@cloud-41:~/tp5$
```

Question 11 Comment réaliser un top-10 ? C'est à dire les 10 premiers fichiers contenant le mot et dont les tf-idf sont les plus hauts

```java
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;

public class Top10TFIDF {

    public static class Top10Mapper extends Mapper<Object, Text, NullWritable, Text> {

        private TreeMap<Double, String> top10Map = new TreeMap<>();
        private String wordToSearch;

        @Override
        protected void setup(Context context) throws IOException, InterruptedException {
            wordToSearch = context.getConfiguration().get("wordToSearch");
        }

        public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
            String[] parts = value.toString().split("\\s+");

            if (parts.length >= 2) {
                String motChemin = parts[0];
                String[] motCheminParts = motChemin.split("\\|");

                if(motCheminParts.length == 2) {
                    String word = motCheminParts[0];
                    String fileName = motCheminParts[1];
                    double tfidf = Double.parseDouble(parts[1]);

                    if (word.equals(wordToSearch)) {
                        top10Map.put(tfidf, fileName);

                        // Conserver les 10 premieres entrees dans le TreeMap
                        if (top10Map.size() > 2) {
                            top10Map.remove(top10Map.firstKey());
                        }
                    }
                }
            }
        }

        protected void cleanup(Context context) throws IOException, InterruptedException {
            // Ecrit les 10 premieres entrees dans le contexte de sortie
            for (String fileName : top10Map.values()) {
                context.write(NullWritable.get(), new Text(fileName));
            }
        }
    }

```

```java

    public static void main(String[] args) throws Exception {
        if (args.length != 3) {
            System.exit(1);
        }

        Configuration conf = new Configuration();
        conf.set("wordToSearch", args[2]);

        Job job = Job.getInstance(conf, "Top10TFIDF");
        job.setJarByClass(Top10TFIDF.class);
        job.setMapperClass(Top10Mapper.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setOutputKeyClass(NullWritable.class);
        job.setOutputValueClass(Text.class);
        TextInputFormat.addInputPath(job, new Path(args[0]));
        TextOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

<u>Résultat</u> : On effectue un top 2 plutôt qu'un top 10 des fichiers, car le dossier rep ne contient que 4 fichiers. On fait le test avec le mot toto qui apparait dans les fichiers 1, 2 et 3 :

```
etudiant@cloud-41:~/tp5$ hadoop jar Top10TFIDF.jar Top10TFIDF /tfidf_4/part-r-00
000 /top10_3 toto
```

Dans le résultat, on obtient bien 2 fichiers, 1 et 2, qui correspondent à ceux avec le tf-idf le plus élevé :

```
etudiant@cloud-41:~/tp5$ hadoop fs -cat /top10_3/part-r-00000
hdfs://cloud-41.depinfo.etu:9000/rep/1
hdfs://cloud-41.depinfo.etu:9000/rep/2
etudiant@cloud-41:~/tp5$
```